HIGH PERFORMANCE COMPUTING

ASSIGNMENT-2

Name : M.Harshith

HTNO : 2303A51444

Batch no : 21

Section 1 : Section A — Vector Operations & Dot Product (CPU- bound, loop overhead)

```
import time
import random
import cProfile
import pstats
import io
import tracemalloc

# ------------------------------------------------
# Vector Operations
# ------------------------------------------------

def vector_addition(a, b):
    result = []
    for i in range(len(a)):
        result.append(a[i] + b[i])
    return result


def dot_product(a, b):
    dot = 0
    for i in range(len(a)):
        dot += a[i] * b[i]
    return dot


# ------------------------------------------------
# Main Execution
# ------------------------------------------------

def main():
    size = 100000  # size of vectors

    # Generate random vectors
    vec1 = [random.random() for _ in range(size)]
    vec2 = [random.random() for _ in range(size)]

    # Measure time for vector addition
```

```python
    start_time = time.time()
    add_result = vector_addition(vec1, vec2)
    add_time = time.time() - start_time

    # Measure time for dot product
    start_time = time.time()
    dot_result = dot_product(vec1, vec2)
    dot_time = time.time() - start_time

    print("Vector Addition Time:", add_time, "seconds")
    print("Dot Product Time:", dot_time, "seconds")
    print("Dot Product Result:", dot_result)


# ----------------------------------------------
# Profiling & Memory Tracking
# ----------------------------------------------

if __name__== "__main__":
    tracemalloc.start()

    profiler = cProfile.Profile()
    profiler.enable()

    main()

    profiler.disable()

    # Capture profiling output
    s = io.StringIO()
    stats = pstats.Stats(profiler, stream=s).sort_stats("cumulative")
    stats.print_stats(10)

    print("\n--- Profiling Results (Top 10) ---")
    print(s.getvalue())

    # Memory usage
    current, peak = tracemalloc.get_traced_memory()
    print(f"Current Memory Usage: {current / 10**6:.2f} MB")
    print(f"Peak Memory Usage: {peak / 10**6:.2f} MB")

    tracemalloc.stop()
```

```
Vector Addition Time: 0.714285135269165 seconds
Dot Product Time: 0.31422996520996094 seconds
Dot Product Result: 24980.977204187504

--- Profiling Results (Top 10) ---
         300151 function calls (300150 primitive calls) in 1.864 seconds
```

```
   Ordered by: cumulative time
   List reduced from 25 to 10 due to restriction <10>

    ncalls  tottime  percall  cumtime  percall filename:lineno(function)
       2/1    0.635    0.318    1.035    1.035 /tmp/ipython-input-
1594222699.py:30(main)
         1    0.443    0.443    0.606    0.606 /tmp/ipython-input-
1594222699.py:12(vector_addition)
         1    0.314    0.314    0.314    0.314 /tmp/ipython-input-
1594222699.py:19(dot_product)
    200000    0.275    0.000    0.275    0.000 {method 'random' of
'_random.Random' objects}
    100000    0.193    0.000    0.193    0.000 {method 'append' of 'list'
objects}
         3    0.000    0.000    0.003    0.001 {built-in method
builtins.print}
        16    0.002    0.000    0.003    0.000 /usr/local/lib/python3.12/dist-
packages/ipykernel/iostream.py:526(write)
        16    0.000    0.000    0.000    0.000 /usr/local/lib/python3.12/dist-
packages/ipykernel/iostream.py:456(_schedule_flush)
         1    0.000    0.000    0.000    0.000 {method 'disable' of
'_lsprof.Profiler' objects}
         1    0.000    0.000    0.000    0.000 /usr/local/lib/python3.12/dist-
packages/ipykernel/iostream.py:202(schedule)



Current Memory Usage: 0.04 MB
Peak Memory Usage: 9.61 MB
```

Section 2: Naïve Matrix Multiplication (O(n3) compute)

```python
import time
import random
import cProfile
import pstats
import io
import tracemalloc

# ---------------------------------------------------
# Naïve Matrix Multiplication
# ---------------------------------------------------
def matrix_multiply(A, B):
    n = len(A)
    m = len(B[0])
    p = len(B)

    # Initialize result matrix
    C = [[0 for _ in range(m)] for _ in range(n)]
```

```python
    # Triple nested loop (O(n^3))
    for i in range(n):
        for j in range(m):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]

    return C


# ----------------------------------------------------
# Main Function
# ----------------------------------------------------
def main():
    size = 50   # Change to 100 or 150 if system is powerful

    # Generate random matrices
    A = [[random.random() for _ in range(size)] for _ in range(size)]
    B = [[random.random() for _ in range(size)] for _ in range(size)]

    # Measure execution time
    start_time = time.time()
    C = matrix_multiply(A, B)
    end_time = time.time()

    print("Matrix Size:", size, "x", size)
    print("Execution Time:", end_time - start_time, "seconds")
    print("Sample Output C[0][0]:", C[0][0])


# ----------------------------------------------------
# Profiling & Memory Tracking
# ----------------------------------------------------
if __name___== "__main__":

    # Start memory tracking
    tracemalloc.start()

    # Start CPU profiling
    profiler = cProfile.Profile()
    profiler.enable()

    # Run main program
    main()

    # Stop profiler
    profiler.disable()

    # Display profiling results
```

```python
    s = io.StringIO()
    stats = pstats.Stats(profiler, stream=s).sort_stats("cumulative")
    stats.print_stats(10)

    print("\n--- Profiling Results (Top 10 Functions) ---")
    print(s.getvalue())

    # Memory usage results
    current, peak = tracemalloc.get_traced_memory()
    print(f"Current Memory Usage: {current / 10**6:.2f} MB")
    print(f"Peak Memory Usage: {peak / 10**6:.2f} MB")

    tracemalloc.stop()
```

```
Matrix Size: 50 x 50
Execution Time: 0.113677978515625 seconds
Sample Output C[0][0]: 12.841882144986402

--- Profiling Results (Top 10 Functions) ---
         5370 function calls (5366 primitive calls) in 0.129 seconds

   Ordered by: cumulative time
   List reduced from 74 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        2    0.000    0.000    0.120    0.060
/usr/lib/python3.12/asyncio/base_events.py:1922(_run_once)
        2    0.003    0.001    0.119    0.060
/usr/lib/python3.12/selectors.py:451(select)
        1    0.001    0.001    0.116    0.116 /tmp/ipython-input-
2467793161.py:31(main)
        1    0.114    0.114    0.114    0.114 /tmp/ipython-input-
2467793161.py:11(matrix_multiply)
     5000    0.007    0.000    0.007    0.000 {method 'random' of
'_random.Random' objects}
        4    0.001    0.000    0.005    0.001 /usr/local/lib/python3.12/dist-
packages/zmq/sugar/socket.py:632(send)
        2    0.000    0.000    0.001    0.001 /usr/local/lib/python3.12/dist-
packages/zmq/eventloop/zmqstream.py:654(_rebuild_io_state)
        3    0.000    0.000    0.001    0.000 {built-in method
builtins.print}
       18    0.001    0.000    0.001    0.000 /usr/local/lib/python3.12/dist-
packages/ipykernel/iostream.py:526(write)
        1    0.000    0.000    0.001    0.001
/usr/lib/python3.12/asyncio/events.py:86(_run)
```

```
Current Memory Usage: 0.08 MB
Peak Memory Usage: 0.27 MB
```

Section 3: 2D Convolution (Stencil/Blur) on a Grid Aim

```python
import time
import cProfile
import pstats
import io
import tracemalloc

# ---------------------------------------------------
# 2D Convolution (5x5 Blur Kernel)
# ---------------------------------------------------
def blur_2d(image, kernel):
    height = len(image)
    width = len(image[0])
    k_size = len(kernel)
    pad = k_size // 2

    # Initialize output image with zeros
    output = [[0.0 for _ in range(width)] for _ in range(height)]

    # Apply convolution (ignore borders)
    for i in range(pad, height - pad):
        for j in range(pad, width - pad):
            acc = 0.0
            for ki in range(k_size):
                for kj in range(k_size):
                    acc += image[i + ki - pad][j + kj - pad] * kernel[ki][kj]
            output[i][j] = acc

    return output


# ---------------------------------------------------
# Main Function
# ---------------------------------------------------
def main():
    size = 200  # Image size (200x200)

    # Create synthetic image
    image = [[(i + j) % 256 for j in range(size)] for i in range(size)]

    # 5x5 Blur Kernel (Uniform)
    kernel = [[1 / 25 for _ in range(5)] for _ in range(5)]

    # Measure execution time
    start_time = time.time()
```

```python
    result = blur_2d(image, kernel)
    end_time = time.time()

    print("Image Size:", size, "x", size)
    print("Execution Time:", end_time - start_time, "seconds")
    print("Sample Output Pixel [100][100]:", result[100][100])


# ----------------------------------------------------
# Profiling & Memory Tracking
# ----------------------------------------------------
if __name__== "__main__":

    tracemalloc.start()

    profiler = cProfile.Profile()
    profiler.enable()

    main()

    profiler.disable()

    # Profiling output
    s = io.StringIO()
    stats = pstats.Stats(profiler, stream=s).sort_stats("cumulative")
    stats.print_stats(10)

    print("\n--- Profiling Results (Top 10 Functions) ---")
    print(s.getvalue())

    # Memory usage
    current, peak = tracemalloc.get_traced_memory()
    print(f"Current Memory Usage: {current / 10**6:.2f} MB")
    print(f"Peak Memory Usage: {peak / 10**6:.2f} MB")

    tracemalloc.stop()
```

```
Image Size: 200 x 200
Execution Time: 2.0604727268218994 seconds
Sample Output Pixel [100][100]: 200.00000000000003

--- Profiling Results (Top 10 Functions) ---
         437 function calls (433 primitive calls) in 2.084 seconds

   Ordered by: cumulative time
   List reduced from 81 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        2    0.000    0.000    2.083    1.042
```

```
/usr/lib/python3.12/asyncio/base_events.py:1922(_run_once)
        2    0.000    0.000    2.077    1.038
/usr/lib/python3.12/selectors.py:451(select)
        2    0.016    0.008    2.077    1.038 {method 'poll' of
'select.epoll' objects}
        1    0.214    0.214    2.061    2.061 /tmp/ipython-input-
1836106658.py:34(main)
        1    1.005    1.005    1.005    1.005 {built-in method time.sleep}
        1    0.841    0.841    0.841    0.841 /tmp/ipython-input-
1836106658.py:10(blur_2d)
        1    0.000    0.000    0.006    0.006
/usr/lib/python3.12/asyncio/events.py:86(_run)
        1    0.000    0.000    0.006    0.006 {method 'run' of
'_contextvars.Context' objects}
        1    0.000    0.000    0.006    0.006 /usr/local/lib/python3.12/dist-
packages/tornado/ioloop.py:750(_run_callback)
        1    0.000    0.000    0.006    0.006 /usr/local/lib/python3.12/dist-
packages/zmq/eventloop/zmqstream.py:685(<lambda>)



Current Memory Usage: 0.07 MB
Peak Memory Usage: 1.61 MB
```

Section 4: Monte Carlo π Estimation (Random + Reduction)

```python
import time
import random
import cProfile
import pstats
import io
import tracemalloc

# ---------------------------------------------------
# Monte Carlo Pi Estimation
# ---------------------------------------------------
def monte_carlo_pi(num_samples):
    inside_circle = 0

    for _ in range(num_samples):
        x = random.random()
        y = random.random()

        if x * x + y * y <= 1.0:
            inside_circle += 1

    pi_estimate = 4 * inside_circle / num_samples
    return pi_estimate
```

```python
# ---------------------------------------------------
# Main Function
# ---------------------------------------------------
def main():
    samples = 5_000_000  # Number of random points

    start_time = time.time()
    pi_value = monte_carlo_pi(samples)
    end_time = time.time()

    print("Number of Samples:", samples)
    print("Estimated Pi Value:", pi_value)
    print("Execution Time:", end_time - start_time, "seconds")


# ---------------------------------------------------
# Profiling & Memory Tracking
# ---------------------------------------------------
if __name__== "__main__":

    tracemalloc.start()

    profiler = cProfile.Profile()
    profiler.enable()

    main()

    profiler.disable()

    # Profiling output
    s = io.StringIO()
    stats = pstats.Stats(profiler, stream=s).sort_stats("cumulative")
    stats.print_stats(10)

    print("\n--- Profiling Results (Top 10 Functions) ---")
    print(s.getvalue())

    # Memory usage
    current, peak = tracemalloc.get_traced_memory()
    print(f"Current Memory Usage: {current / 10**6:.2f} MB")
    print(f"Peak Memory Usage: {peak / 10**6:.2f} MB")

    tracemalloc.stop()
```

```
Number of Samples: 5000000
Estimated Pi Value: 3.1411456
Execution Time: 12.97388243675232 seconds
```

```
--- Profiling Results (Top 10 Functions) ---
         10000154 function calls (10000153 primitive calls) in 12.975 seconds

   Ordered by: cumulative time
   List reduced from 24 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      2/1    0.726    0.363   12.975   12.975 /tmp/ipython-input-
1834780482.py:28(main)
       12   10.156    0.846   12.063    1.005 {built-in method time.sleep}
 10000000    2.046    0.000    2.046    0.000 {method 'random' of
'_random.Random' objects}
        1    0.047    0.047    0.055    0.055 /tmp/ipython-input-
1834780482.py:11(monte_carlo_pi)
        3    0.000    0.000    0.001    0.000 {built-in method
builtins.print}
       14    0.000    0.000    0.001    0.000 /usr/local/lib/python3.12/dist-
packages/ipykernel/iostream.py:526(write)
       14    0.000    0.000    0.001    0.000 /usr/local/lib/python3.12/dist-
packages/ipykernel/iostream.py:456(_schedule_flush)
        1    0.000    0.000    0.001    0.001 /usr/local/lib/python3.12/dist-
packages/ipykernel/iostream.py:202(schedule)
        1    0.000    0.000    0.000    0.000 /usr/local/lib/python3.12/dist-
packages/zmq/sugar/socket.py:632(send)
        1    0.000    0.000    0.000    0.000 {method 'disable' of
'_lsprof.Profiler' objects}



Current Memory Usage: 0.02 MB
Peak Memory Usage: 0.02 MB
```

Section 5: Pairwise Interactions (N2 Kernel, “Mini” N-Body)

```python
import time
import random
import cProfile
import pstats
import io
import tracemalloc
import math

# ---------------------------------------------------
# Pairwise Interaction Kernel (N^2)
# ---------------------------------------------------
def pairwise_interactions(particles):
    n = len(particles)
    total_interaction = 0.0

    for i in range(n):
```

```python
        xi, yi = particles[i]
        for j in range(i + 1, n):
            xj, yj = particles[j]

            # Compute distance (no physics integration)
            dx = xi - xj
            dy = yi - yj
            distance = math.sqrt(dx * dx + dy * dy) + 1e-9

            total_interaction += 1.0 / distance

    return total_interaction


# ---------------------------------------------------
# Main Function
# ---------------------------------------------------
def main():
    N = 2000  # Number of particles

    # Generate random particle positions
    particles = [(random.random(), random.random()) for _ in range(N)]

    # Measure execution time
    start_time = time.time()
    interaction_value = pairwise_interactions(particles)
    end_time = time.time()

    print("Number of Particles:", N)
    print("Total Interaction Value:", interaction_value)
    print("Execution Time:", end_time - start_time, "seconds")


# ---------------------------------------------------
# Profiling & Memory Tracking
# ---------------------------------------------------
if __name__== "__main__":

    tracemalloc.start()

    profiler = cProfile.Profile()
    profiler.enable()

    main()

    profiler.disable()

    # Profiling output
```

```python
    s = io.StringIO()
    stats = pstats.Stats(profiler, stream=s).sort_stats("cumulative")
    stats.print_stats(10)

    print("\n--- Profiling Results (Top 10 Functions) ---")
    print(s.getvalue())

    # Memory usage
    current, peak = tracemalloc.get_traced_memory()
    print(f"Current Memory Usage: {current / 10**6:.2f} MB")
    print(f"Peak Memory Usage: {peak / 10**6:.2f} MB")

    tracemalloc.stop()
```

```
Number of Particles: 2000
Total Interaction Value: 5990982.9975241935
Execution Time: 9.654498815536499 seconds

--- Profiling Results (Top 10 Functions) ---
         2003607 function calls (2003597 primitive calls) in 9.674 seconds

   Ordered by: cumulative time
   List reduced from 109 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        4    0.000    0.000    9.664    2.416
/usr/lib/python3.12/asyncio/base_events.py:1922(_run_once)
        1    0.012    0.012    9.638    9.638 /tmp/ipython-input-
3511488703.py:34(main)
        9    7.744    0.860    9.053    1.006 {built-in method time.sleep}
  1999000    1.395    0.000    1.395    0.000 {built-in method math.sqrt}
        1    0.488    0.488    0.571    0.571 /tmp/ipython-input-
3511488703.py:12(pairwise_interactions)
      3/2    0.000    0.000    0.024    0.012 {method 'run' of
'_contextvars.Context' objects}
        2    0.000    0.000    0.024    0.012 /usr/local/lib/python3.12/dist-
packages/zmq/eventloop/zmqstream.py:574(_handle_events)
        1    0.000    0.000    0.024    0.024 /usr/local/lib/python3.12/dist-
packages/tornado/platform/asyncio.py:206(_handle_events)
        2    0.000    0.000    0.023    0.012 /usr/local/lib/python3.12/dist-
packages/zmq/eventloop/zmqstream.py:615(_handle_recv)
        2    0.000    0.000    0.023    0.011 /usr/local/lib/python3.12/dist-
packages/zmq/eventloop/zmqstream.py:547(_run_callback)



Current Memory Usage: 0.15 MB
Peak Memory Usage: 0.20 MB
```